



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

Advanced Database Systems (INFS3200)

Lecture 13: Course Review

Lecturer: Teaching Team

School of Electrical Engineering and Computer Science (EECS)

Faculty of Engineering, Architecture and Information Technology

The University of Queensland

Outline

Course Revision

- Final Exam - when, how, and what!
- Topic Review

Final Exam Information

INFS3200 Final Exam (on-campus)

- Time: **5:45 PM AEST, 10 June 2025**
- Examination Duration: 120 minutes
- Reading Time: 10 minutes (planning)
- Closed book (No materials permitted)
 - On-campus Exam: Central Paper-based Exam
- Authorised Calculators are allowed
- Short Answer Questions ONLY
- Check SI-net for the official exam time & other information

Final Exam Information

Exam Scope (50%)

- Covering the entire course materials
- Focusing on the Lecture Notes and Tutorial Questions
 - ❖ Examples in tutorials and lectures are important
 - ❖ No practicals

Survive Guide

- Reasoning + Questioning
- Tutorial Questions (★★★★)
- Past Exam Papers (★★★★)
- Questions/Samples discussed during the lectures of Weeks 2-12 (★★)

Final Exam Preparation

The short answer questions

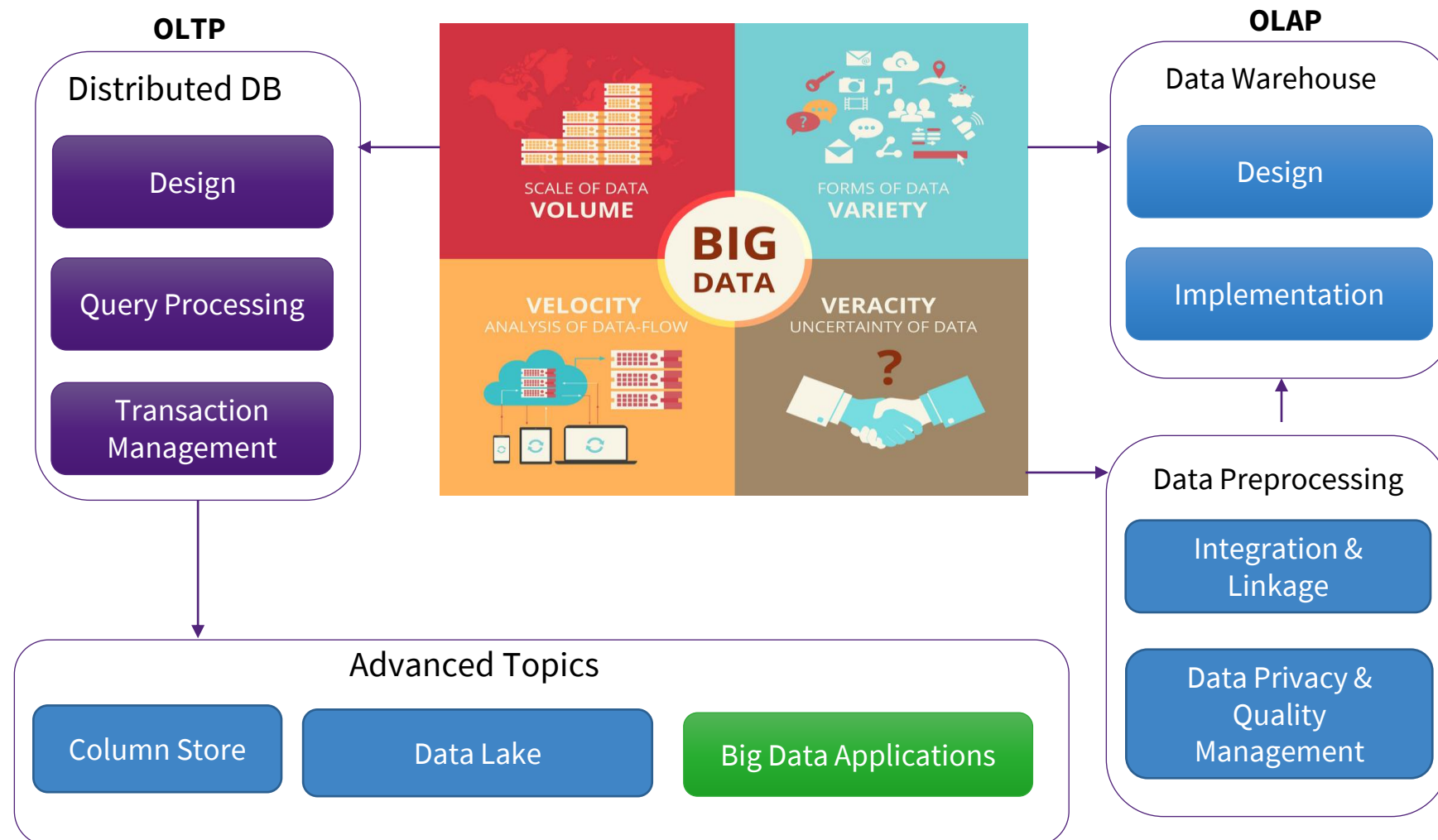
- ❖ No multiple-choice questions
- ❖ Mainly for brief discussions
- ❖ Short answers with manually calculated results
- ❖ Problem-solving + simple calculation + concepts

The same style as past exams (2020 – 2023, Sem1 & Sem2 2024)

- ❖ Past exam questions could re-appear
- ❖ You may be able to pass the exam if you have tried past exam questions without much difficulty.

<https://web.library.uq.edu.au/library-services/students/past-exam-papers>

Course Structure



INFS3200 Modules

Distributed Databases

- DDB Concepts
- DDB Design
- DDB Query Processing
- DDB Transaction Management

Data Warehousing

- DW Design
- DW Implementation

Column-Oriented Databases

Data Integration and Linkage

Data Privacy and Quality Management

Data Lake

Advanced Database Applications

Final Exam Content:

There are eight parts to be examined

Part 1: Distributed Databases - Junliang

Part 2: Distributed Query Processing - Junliang

Part 3: Distributed Transaction Management - Junliang

Part 4: Data Warehouse - Shazia

Part 5: Column Oriented Databases – Zhi/Sen

Part 6: Database Integration and Data Linkage – Sen

Part 7: Data Privacy and Quality Management - Sen

Part 8: Data Lake and Advanced Topics – Zhi/Sen

Part 1: Distributed Databases

DDB Concepts

- Scale Up vs. Scale Out
- Top-Down Design
- Distribution, Homogeneity, Autonomy
- SSOT (Single Source of Truth) problem

Fragmentation, replication and allocation

- Horizontal, vertical and mixed fragmentation
- Primary and derived horizontal fragmentation

Fragmentation property (design criteria)

- **Completeness**
- **Disjointness**
- **Reconstruction**

Tools for fragmentation

- *Minterm* predicates and their application in the horizontal fragmentation process

What is a good fragmentation?

- **Completeness:** $\forall t \in R, \exists F_i \in F: t \in F_i$
- **Disjointness:** $\forall F_i, F_j \in F, i \neq j \Rightarrow F_i \cap F_j = \phi$
- **Reconstruction:** $R = \bigcup_{i=1, \dots, n} F_i$

Sample Question

WID	WName	Country
W01	Delta Depot	USA
W02	Omega Vault	Germany
W03	Sigma Store	Germany
W04	Alpha Point	Japan

SID	Item	Quantity	WID
5001	Steel	100	W01
5002	Copper	150	W02
5003	Glass	200	W03
5004	Rubber	90	W01
5005	Plastic	160	W03
5006	Lithium	130	W04

A company maintains two relations in its distributed logistics database: WID is the primary key.

- The company wants to horizontally fragment the Warehouse relation using the predicate-based fragmentation:

f_1 : Country = 'USA'

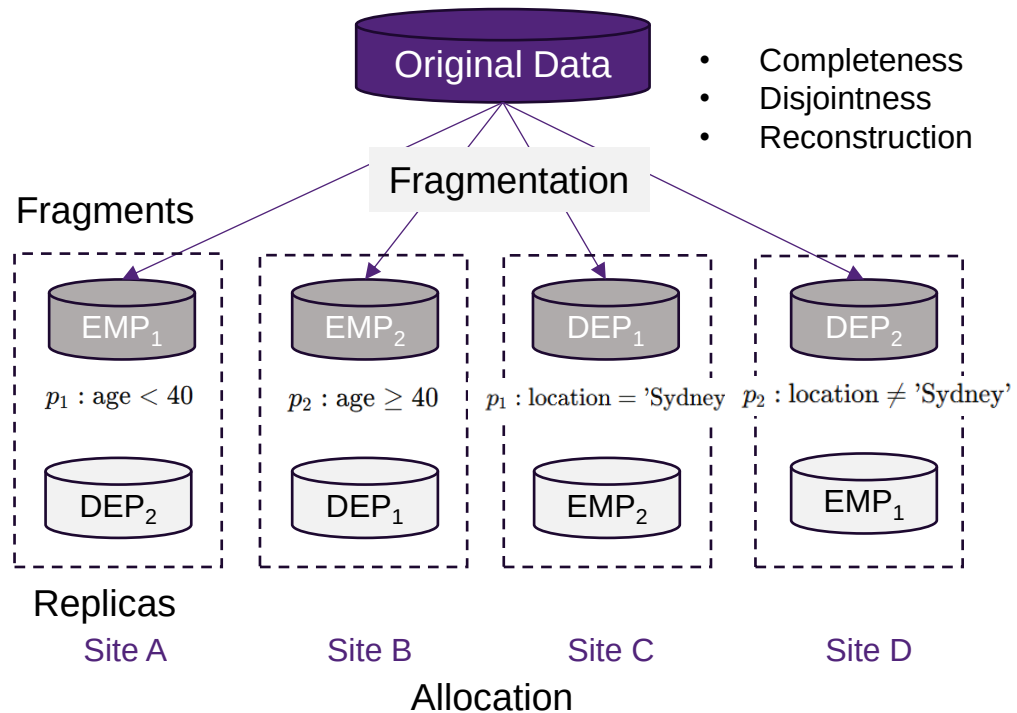
f_2 : Country = 'Germany'

f_3 : Country = 'Japan'

Now apply derived horizontal fragmentation to Shipment based on this fragmentation of Warehouse. Show the resulting Shipment fragments.

- Suppose these derived fragments are named sh_1, sh_2, and sh_3. Each fragment is replicated as rp_1, rp_2, and rp_3 for fault tolerance. How many correct combinations can be derived? Justify your conclusion.
- Why is fragmentation / replication needed? What makes for a good allocation?
- Given a few fragments, can you identify if CDR properties are satisfied?

Review



Query

SELECT E.name FROM EMP E, DEP D
WHERE E.dept_id = D.dept_id AND D.location = 'Sydney'

Global Query Decomposition

$$\pi_{E.name} (\sigma_{D.location = 'Sydney'} (EMP \bowtie_{E.dept_id=D.dept_id} DEP))$$

Rewrite the query (Normalization)

Data Localization

$$\pi_{name} (\sigma_{location = 'Sydney'} ((EMP_1 \cup EMP_2) \bowtie (DEP_1 \cup DEP_2)))$$

Replace original data with fragments

Reduction in Data Localization

Eliminate redundant fragments $\pi_{name} (\sigma_{location = 'Sydney'} ((EMP_1 \cup EMP_2) \bowtie DEP_1))$

$$\pi_{name} ((EMP_1 \bowtie_{\sigma_{location = 'Sydney'}} (DEP_1)) \cup (EMP_2 \bowtie_{\sigma_{location = 'Sydney'}} (DEP_1)))$$

Push down selection Push up join and union

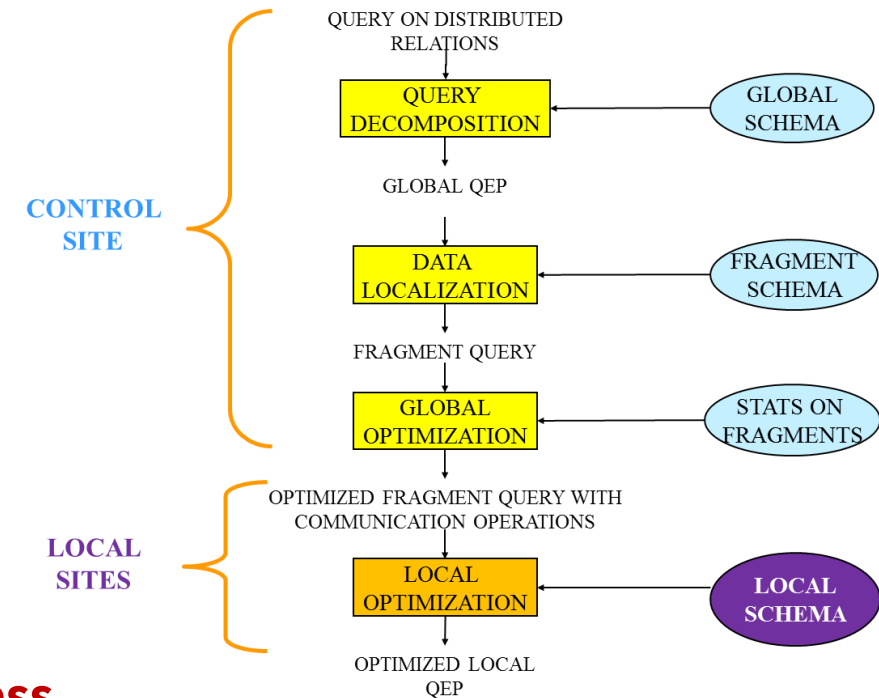
Global Query Optimization

$$\pi_{name} (EMP_1 \bowtie_{dept_id} DEP_1) \cup \pi_{name} (EMP_2 \bowtie_{dept_id} DEP_1)$$

Adjust join ordering and use semi-join

Part 2: Distributed Query Processing

- Why DQP?
- **Layers of QP and the functions of each layer.**
- **Search Strategies and Global Query Optimization**
- Search Space vs. Solution Space
- Cost Functions
- **Reduction Techs in Data Localization**
- **Join Ordering**
- **Semi-join operation with multiple sites and its detailed process**
 - Why is Semi-join efficient? ==> transfer cost



Q2. DQP

In addition to the lecture slides, please thoroughly review THREE questions in T4.
For Example:

Consider the following database (arrows are foreign-key linkages): (Question 2 in T4)

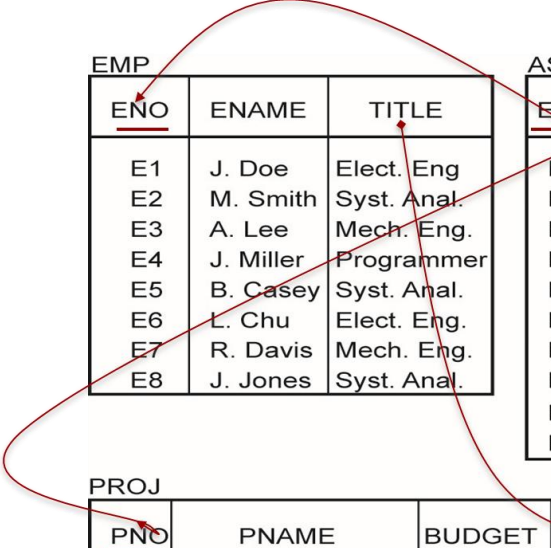
(a) Assume that relation PROJ is horizontally fragmented as follows:

$$\text{PROJ1} = \sigma_{\text{PNO} \leq "P2"} \text{PROJ}$$

$$\text{PROJ2} = \sigma_{\text{PNO} > "P2"} \text{PROJ}$$

Transform the following query into **a reduced query** on fragments:

```
SELECT ENO, PNAME
FROM PROJ, ASG
WHERE PROJ.PNO = ASG.PNO AND PNO = "P4"
```



ENO	ENAME	TITLE
E1	J. Doe	Elect. Eng
E2	M. Smith	Syst. Anal.
E3	A. Lee	Mech. Eng.
E4	J. Miller	Programmer
E5	B. Casey	Syst. Anal.
E6	L. Chu	Elect. Eng.
E7	R. Davis	Mech. Eng.
E8	J. Jones	Syst. Anal.

ENO	PNO	RESP	DUR
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36
E8	P3	Manager	40

PNO	PNAME	BUDGET
P1	Instrumentation	150000
P2	Database Develop.	135000
P3	CAD/CAM	250000
P4	Maintenance	310000

TITLE	SAL
Elect. Eng.	40000
Syst. Anal.	34000
Mech. Eng.	27000
Programmer	24000

Part 3: Distributed Transaction Management

Transaction Management in centralized DBMS

- **ACID Properties of database transactions**
- **Concurrency Control**
 - **Conflicting operations, problems and 2PL**
- Durability and Recovery
- **Database State**

Distributed Transaction Management

- Concurrency Control and Recovery in Distributed Databases
- **Two phase Commit Protocol**
- **Maintaining Replicas in Distributed Databases**

Part 3: Distributed Transaction Management

Serial Schedule (**non interleaving**) (p 18 in L3)

T1	T2
read-item (X); X:=X-N; write-item (X); read-item (Y); Y:=Y+N; write-item (Y);	 read-item (X); X:=X+M; write-item (X);

- Binary Locks
- Read/Write Locks
- Two-Phase Locking (2PL)

Non-Serial Schedule (**Interleaving**)

T1	T2
read-item (X); X:=X-N; write-item (X);	 read-item (X); X:=X+M; write-item (X);
 read-item (Y); Y:=Y+N; write-item (Y);	

System Log:

A journal keep track of all transactions

- REDO/UNDO,..
- Write Ahead Logging
 - Force-write the log before data is written
 - Write logs before commit

“Non-Serial Schedule”: Schedule with interleaved operations.

Problem in concurrency control?

Conflicting Operations

Operations of a schedule are in **conflict** if they meet all the following three conditions at the same time:

- they belong to different transactions
- they access the same item X
- at least one is a write-item (X)

A **Write-Write** conflict occurs when two transactions try to write different values to the same item. The final value depends on which write happens last, potentially leading to **lost updates**.

A **Read-Write** conflict arises because one transaction might read outdated data or uncommitted data (leading to **dirty read**).

Given two transactions with the following concurrent operations.

Conflicting
operations

Case 1:	r1(X); w2(X)
Case 2:	r1(X); w1(X)
Case 3:	r1(X); r2(X)
Case 4:	w1(X); w2(Y)
Case 5:	w1(X); w2(X)
Case 6:	w1(Y); r2(Y)

Two-Phase Commit (2PC) Protocol

If coordinator gets **unanimous** yes votes:

- The coordinator writes a commit log record and sends commit message to all subordinates

Else

- writes abort log rec, and sends abort message

Subordinates write abort/commit log record based on the message they get, then send **ack** message to coordinator

Coordinator writes end log record after getting all **acks**

Updating Distributed Data

Synchronous Replication (Eager approach)

- “All copies” of a modified relation (fragment) must be updated before the modifying transaction commits
- Data distribution is made **transparent** to users

Asynchronous Replication (Lazy approach)

- Copies of a modified relation are only **periodically** updated; different copies may get out of sync in the meantime
- Users must be **aware of data distribution**
- Most products follow this approach

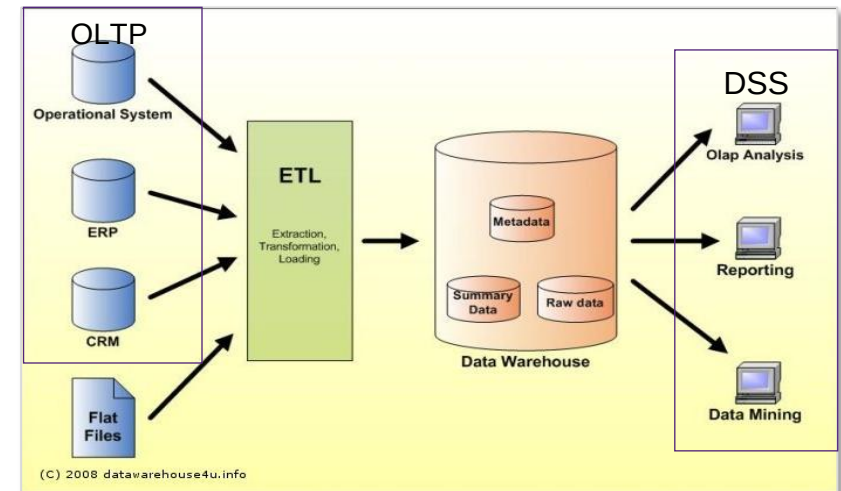
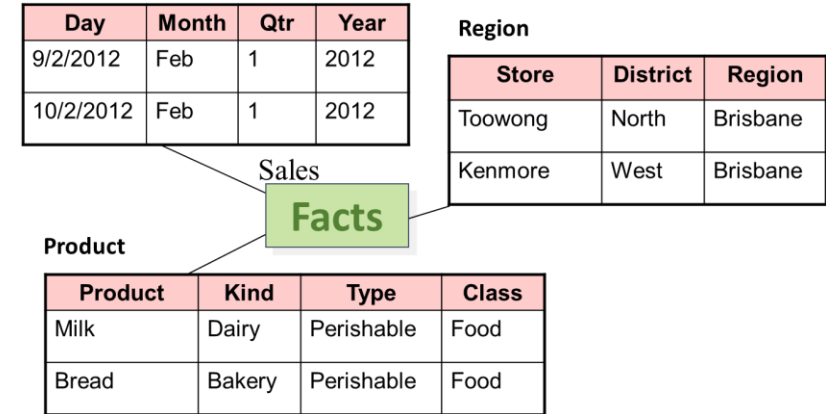
Part 4: Data Warehouse

Motivation and Requirements (pp 5-18 in L5)

- Volume & velocity vs. value
- RDBMS vs. DW
- Overview of DW development (OLTP -> ETL -> DW -> DSS)

Multidimensional Data Model (pp 19-33 in L5)

- Facts and dimensions
- Star schema vs. snowflake schema
- Fact Constellation (a variation of Star Schema)
- The differences between these types of schemas



Part 4: Online Analytical Processing

OLAP Operations (pp 4-47 in L5 & p 9 in L6)

- [Main differences between OLTP and OLAP?](#) (p 4 in L6)
- Example and the meanings of *Drill-down/Roll-up*, *Slicing/Dicing*, *Pivoting*, *CUBE queries*
- What are the results of OLAP operations?
- Applicability of OLAP operations Indexing (pp 12-16 in L6)

Data Warehouse Performance

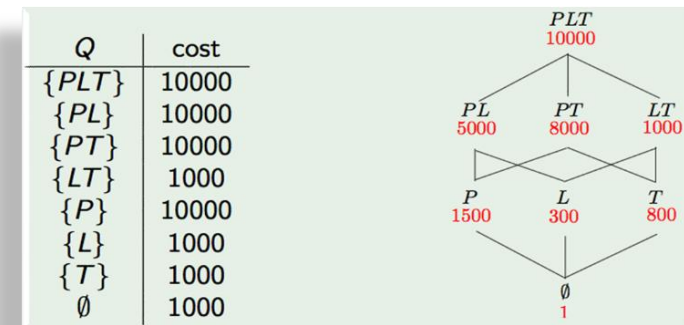
- Bitmap indexing
- Join indexing

Data Warehouse Maintenance & View materialization (pp 17-32 in L6)

- Query with materialized views
- Concepts of *Cuboid*, Lattice
- [Cost vs. Benefit](#) of materialized views
 - Number of possible views to be materialized
 - The number of *cuboids* that can be created by using GROUP BY clause.
 - Which queries are benefitted from a certain view
- Greedy algorithm (pp 33-34 in L6)
 - Calculating the cost and benefit based on the Lattice of Materialized Views

Materialized View Problem:

Given a **fact table T**, a set of views $T_G = \{T_{G1}, T_{G2}, \dots, T_{Gt}\}$ can be defined over T (t is the total number of GROUP BY views definable over T). For a given integer k, $k \leq t$, **find a set S**, $|S| = k$ of cuboids ($S \subseteq T_G$), such that S can be materialized **with the largest benefit(S)**.



The **red** number of each node shows the sorting cost, if this view (node) is materialised.

Q4. Preparing for DW and OLAP Module

Data Warehouse

- Difference between RDBMS and DW
- Challenges in Building a DW
- DW Design and Multidimensional Data Model
- DW Design

Online Analytical Processing

- Operations
- Performance
- Maintenance

In addition to the lecture slides, please thoroughly review **FOUR** questions in T4 and **THREE** questions in T5.

Part 5: Column Oriented Databases

Data Orientation (pp 8 in L8)

- Data orientation refers to how tabular data is represented in a linear memory model such as in-disk or in-memory.
- The two most common representations are column-oriented (columnar format) and row-oriented (row format)
- ✓ The choice of data orientation is a trade-off and an architectural decision in databases, query engines, and numerical simulations

column 1	column 2	column 3
item 11	item 12	item 13
item 21	item 22	item 23

item 11 | item 21 | item 12 | item 22 | item 13 | item 23

column-oriented

item 11 | item 12 | item 13 | item 21 | item 22 | item 23

row-oriented

Part 5: Column Oriented Databases

Definition of column-oriented databases (pp 13 in L8)

- A column-oriented database, also known as columnar database, is a type of database management system (DBMS) that **stores data in columns together** on disk. This enables faster queries for data analytics, which generally involves **filtering** and **aggregating** table columns.



Part 5: Column Oriented Databases

Advantages of column-oriented databases (pp 19-20 in L8)

1. Improved data compression

- Data within a single column is typically **homogenous (same data type)**.
- Amenable to applying advanced compression techniques, reduce storage requirements and associated costs.
- This compression also results in less I/O overhead, as less data needs to be read from storage.

2. Enhanced query performance

- In a columnar database, **only the columns relevant to a query need to be accessed and processed**.
- Selective data retrieval leads to faster query performance.
- Analytical queries typically aggregate or scan large volumes of data.

3. Efficient use of cache memory

- Columnar storage aligns well with **modern processor cache design**. Since columns are stored contiguously, a single cache load can retrieve a large block of relevant data, reducing cache misses and enhancing CPU efficiency.

4. Vectorization and parallel processing

- Columnar databases are well-suited for **vectorized operations**, where the same operation is applied to multiple data points simultaneously.
- This aligns well with modern central processing units architectures and allows for efficient parallel processing, further speeding up query execution.

Part 5: Column Oriented Databases

Advantages of column-oriented databases (pp 19-20 in L8)

5. Improved analytics and reporting

- Inherently efficient at **aggregating and summarizing** data, operations that are fundamental to analytics and reporting.
- They allow for faster calculations and aggregations across vast datasets, ideal for business intelligence and analytical applications.

6. Better handling of sparse data

- Handle **missing or null values** efficiently by simply not storing any data for those missing values

7. Flexible indexing options

- Columnar databases offer flexible **indexing strategies**. Because each column is stored independently, different indexing techniques can be applied to different columns, depending on the nature of the data and the typical queries performed.

8. Ease of scalability

- Generally easier to **scale horizontally** by adding more servers to handle increased load.
- Particularly beneficial in cloud computing environments where resources can be dynamically adjusted based on demand.

9. Real-time data analytics and updates

- Some modern columnar databases have evolved to allow for real-time analytics and high-speed data ingestion.
- Traditional columnar databases were seen as less suitable for transactional systems.

Part 5: Column Oriented Databases

Column-Oriented Database Compression (pp 27-39 in L8)

- **Dictionary Encoding**
 - Dictionary encoding works well for distributions with ***a few very frequent values***, and can also be applied to strings
- **Run-length Encoding**
 - Run-length encoding (RLE) compresses ***runs of the same value*** in a column to a compact singular representation.
- **Delta Encoding**
 - Storing the **difference between consecutive values** rather than the actual values. This method is particularly useful for compressing data with many consecutive values, such as timestamp data types
- **Bit-Packing Encoding**
 - Finds the common high bits of 64 values and trims them, only recording the changing parts.

Part 5: Column Oriented Databases

Delta Encoding (pp 34-35 in L8)

- Storing the **difference between consecutive values** rather than the actual values. This method is particularly useful for compressing data with many consecutive values, such as timestamp data types.

Raw

170

270

270

350

351

351

Column data

Delta
Encoding

170

100

0

130

1

0

Part 5: Column Oriented Databases

Delta Encoding (pp 34-35 in L8)

- Storing the **difference between consecutive values** rather than the actual values. This method is particularly useful for compressing data with many consecutive values, such as timestamp data types.
 - 1. **Initialize** a variable to store the previous value: Initialize a variable to store the previous value, which will be used to calculate the delta.
 - 2. **Traverse**: Traverse the data and for each value, calculate the delta by subtracting the previous value from the current value.
 - 3. **Replace** values with delta: Replace each value in the data with its corresponding delta.
 - 4. **Store** the initial value and the compressed data: Store the initial value and the compressed data (i.e the data with values replaced by deltas) separately.
 - 5. **Decompression**: For decompression, use the stored initial value and the delta to calculate the original value.

Part 5: Column Oriented Databases

Late Materialization (pp 50-58 in L8)

- More recent column-stores choose to ***keep data in columns until much later into the query plan***, operating directly on these columns. In order to do so, intermediate “position” lists often need to be constructed in order to match up operations that have been performed on different columns.

Q5. Sample Questions

In addition to the lecture slides, please thoroughly review THREE questions (No Q4) in T6.

For Example:

- Q1: Prepare a column-oriented representation for the following row-oriented table.

TranID	TranDate	ProductCode	Category	CustNo	Qty	TotalAmt	Discount
7582	2023-08-01	GT4235	Media	V899	7	350.00	10%
7431	2023-08-01	HT8865	Food	N776	24	150.00	0%
9921	2023-08-01	RT5412	Gadget	V899	5	250.00	10%
4143	2023-08-02	GT1245	Media	N998	100	1200.00	0%
8876	2023-08-02	TR554	Food	V665	12	24.00	10%
9900	2023-08-02	GT4235	Media	V665	50	2500.00	10%
6548	2023-08-02	HT8865	Food	V665	24	150.00	10%

Give an example of an SQL query that will perform 'significantly' better with the column-oriented representation.

- Q2: Match the following compressions for the most suitable column in the provided example in Q1: a) Dictionary Encoding b) Bit-Packing Encoding -T64 c) Delta Encoding Conduct the respective compressions and explain why the match you propose is appropriate.
- Q3: Explain how query performance is optimized using: a) Sparse Primary Index b) Materialised Views c) Projections

Part 6: Data Integration and Linkage

Motivation and Challenges in DB Integration (pp 6, 24 in L9)

- Schema heterogeneity
- Data type heterogeneity
- Value heterogeneity
- Semantic heterogeneity

Dynamic programming for
calculating the edit distance

Federated Database (FDB), Multi-Databases (MDB) and Interoperable Information Systems (*Distribution vs. Homogeneity, Autonomy*) (pp 11-13 in L9)

Schema Mapping (pp 26-28 in L9)

Data linkage (pp 47, 50-70)

- The problem of data linkage (i.e., computing the same real-world entity)
- **Distance Measures:** *Edit distance (dynamic programming), Q-gram (n-gram) and Jaccard coefficient, TF/IDF and cosine similarity, Numeric similarity, Phonetic similarity*
- The applicability of different kinds of **similarity computations** – ***similar questions in the previous exams***

Q6. Calculate similarity btw TWO strings using the Jaccard Coefficient (with 3-gram) and Edit Distance

In addition to the lecture slides, please thoroughly review FIVE questions in T5.

For Example:

- A = “*Government of Queensland*”, B = “*Queensland Government*”

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

- ED(“Dubios”, “Dubose”) ?
 - $E[i, j] = [i-1, j-1]$ if $r_i = s_j$
 - $E[i, j] = \min\{E[i, j-1]+1, E[i-1, j-1]+1, E[i-1, j]+1\}$ if $r_i \neq s_j$
 - How to conduct the edit operations (show edit steps)

Q6. Hint

A = “*Government of Queensland*”, B = “*Queensland Government*”

Step 1. list all combinations of q-gram

Step 2. Count overlapped q-grams

Step 3. Calculate J Similarity

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

A

Gov, ove, ver, ...

B

Que, uee, een, ...

Q4. Hint

ED(“Dubios”, “Dubose”) ?

- $E[i, j] = [i-1, j-1]$ if $r_i = s_j$
- $E[i, j] = \min([i, j-1]+1, [i-1, j]+1, [i-1, j-1]+1)$ if $r_i \neq s_j$

		D	U	B	O	S	E
	0	1	2	3	4	5	6
D	1	0	1	2	3	4	5
U	2	1	0	1	2	3	4
B	3	2	1	0	1	2	3
I	4	3	2	1	1	2	3
O	5	4	3	2	1	2	3
S	6	5	4	3	2	1	2

Differences between Edit Distance and q-gram approaches?

- e.g. when does JD work better than edit distance?

[Goto DP](#)

Part 7: Data Privacy and Quality

Data Privacy

- k -anonymity (pp 23-35 in L10)
- l -diversity (pp 36-39 in L10)
- t -closeness (pp 40-42 in L10)
- **Differential Privacy** (pp 43-62 in L10)
 - **Deniability by randomization**
 - Laplace mechanism (Laplace noise with $\lambda = \Delta f / \epsilon = 1 / \epsilon$)

Advantages and Limitations of the above methods

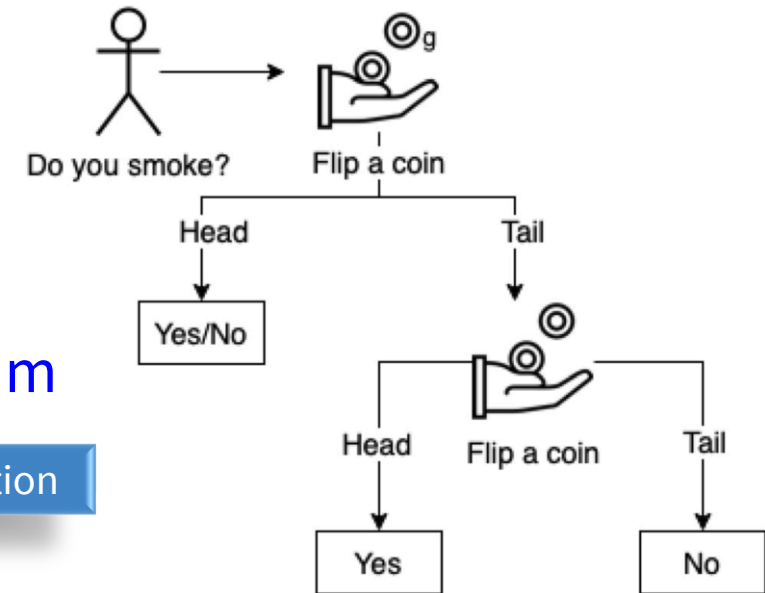
- **Advantages of Differential Privacy** approach over the k -anonymity method.
- What changes will be performed on a database in order to apply the above-mentioned privacy preservation methods?

Data Quality

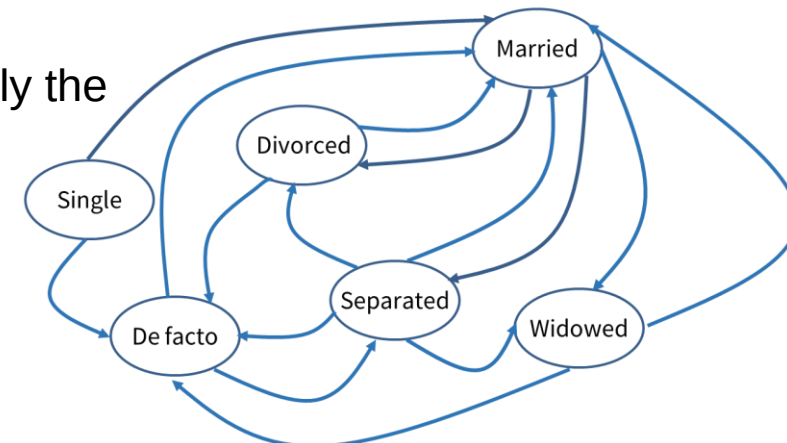
- Measure dimensions (p 66 in L10)
- **Dynamic Data Integrity** (p 68 in L10)

Coin Toss Algorithm

Deniability by Randomization



Question: Can we modify one's marriage status in a database without considering this pattern?



Q7. Sample Questions

In addition to the lecture slides, please thoroughly review THREE questions (No Q4) in T8.

For Example:

- Q1: What is a quasi-identifier? Can a dataset have no quasi-identifier or multiple quasi-identifiers? What is the role a quasi-identifier plays in the k-anonymity approach?
- Q2: How can you protect data privacy (i.e., not revealing an individual's salary) using k-anonymity, l-diversity and t-closeness? Please also discuss their potential problems.
- Q3: Explain the following data quality dimensions with examples: Integrity, Accuracy, Representational consistency, Completeness, etc.

Besides, there are a few questions related to data quality and privacy in the previous exams.

For Example:

- How can the differential privacy approach protect data privacy? (pp 43-62)
- What are the applications of differential privacy techniques? Name a few.
- In data quality issues, what is Dynamic Data Integrity? Briefly explain this concept and give an example.
- Etc.

Part 2

Part 8: Data Lake and Advanced Topics

What is data lake?

- A **flexible**, **scalable** data storage and management system that ingests and stores **raw** data from **heterogeneous** sources in their **original** format.

What motivates data lake?

- Characters of Big data:
 - Semi-structured and unstructured data
 - High volume, high velocity, high variety, veracity issues
 - Heterogeneous source

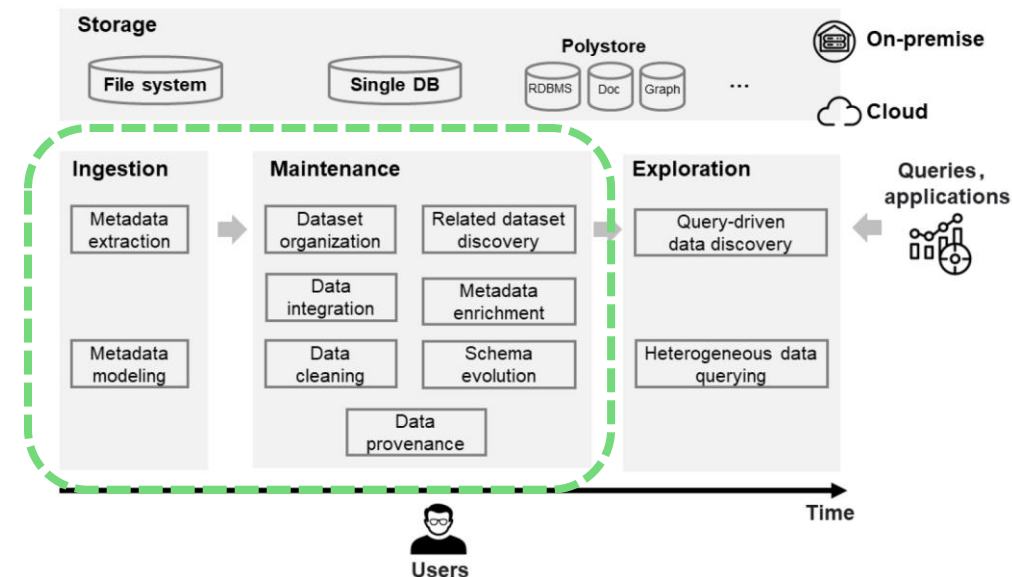
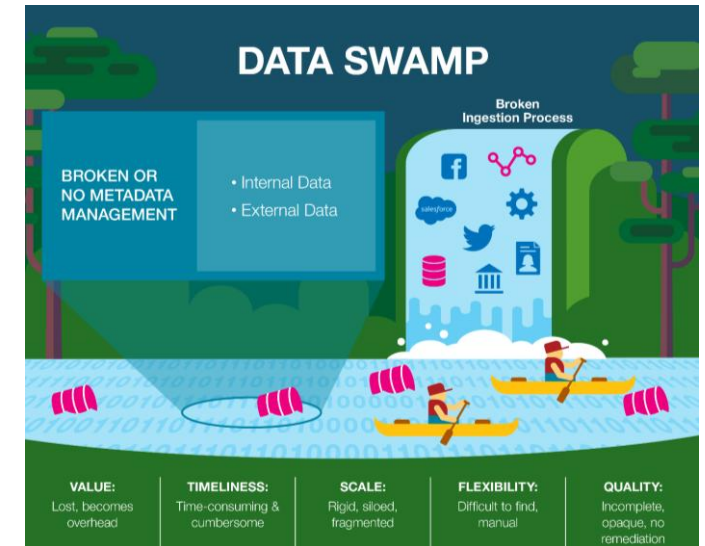
Comparison with RDBMS

- Schema-on-write vs. schema-on-read
- ETL vs. ELT

Part 8: Data Lake and Advanced Topics

“Data lake” or “Data swamp”?

- Key procedures to prevent
 - Ingestion: metadata extraction, metadata modelling
 - What is metadata?
 - Maintenance: dataset organization, data integration, data cleaning, metadata enrichment, data provenance, related dataset discovery, schema evolution



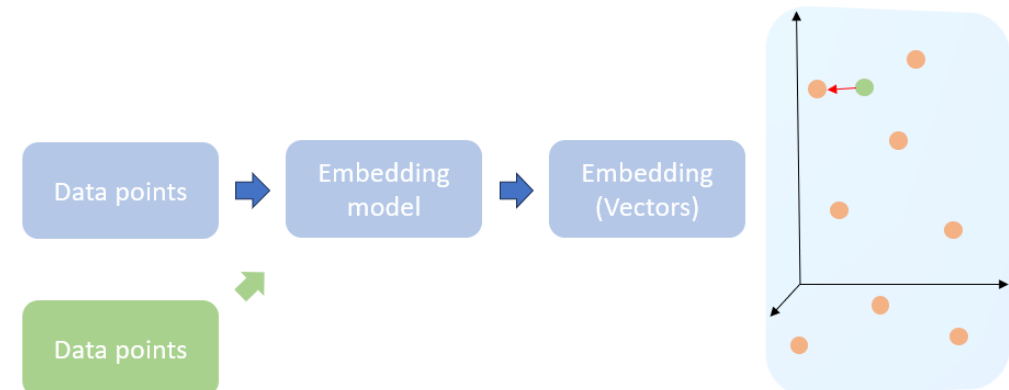
Part 8: Data Lake and Advanced Topics

Big Data is of Three Utilities:

- Connecting Dots – “*From small to big*”
- Discovering Specifics – “*From big to small*”
- Data Inferencing – “*Knowing unknown*”
 - *Dimensionality – Curse or Blessing?*

Vector Database:

- Specialized database system designed to store, manage, and search high-dimensional vectors.
- Some key concepts:
 - Embedding model
 - Embedding
 - Data
 - Query



Part 8: Data Lake and Advanced Topics

NoSQL

- An umbrella term for databases /data stores which do not comply with traditional DBMS principles
- Key differences between SQL and NoSQL:
 - No Schema
 - Flexible access
- Four types of NoSQL database:
 - Document databases
 - Key-value stores
 - Wide-column stores
 - Graph databases

Q8. Data Lake and Advanced Topics

Questions related to the concepts/topics in the lecture:

Data Lake:

- Definition and architecture (pp 8, 18 in L11)
- What measures can be taken to manage a data lake efficiently and prevent it from turning into a data swamp? Please outline two key strategies briefly.

Vector Database:

- What is a vector database? (p 51 in L12)
- Nearest Neighbour search vs approximate Nearest Neighbour search. (pp 59-60 in L12)
- Dimensionality - blessing or curse (pp 27-29 in L12)
 - Benefits of high-dimensional data

NoSQL:

- What is a NoSQL database? Types of NoSQL? (pp 40-42 in L12)
- In-memory key-value stores (pp 46-47 in L12)
- NoSQL vs SQL (p 50 in L12)

Any Questions?

Thanks for listening and studying with us through the whole semester



Wish you all the best in the upcoming Final!!